DF Backend Database Architecture

Version generated: 2026-03-19 00:41

1. Architecture Overview

The database is designed in two layers: **(a)** operational entities for platform management (users, branches, customers), and **(b)** pricing-domain entities for laundry rate-card modeling (categories, products, services, pricing). The pricing domain uses a normalized many-to-many structure between products and services through the pricing table.

2. Core Relationship Model

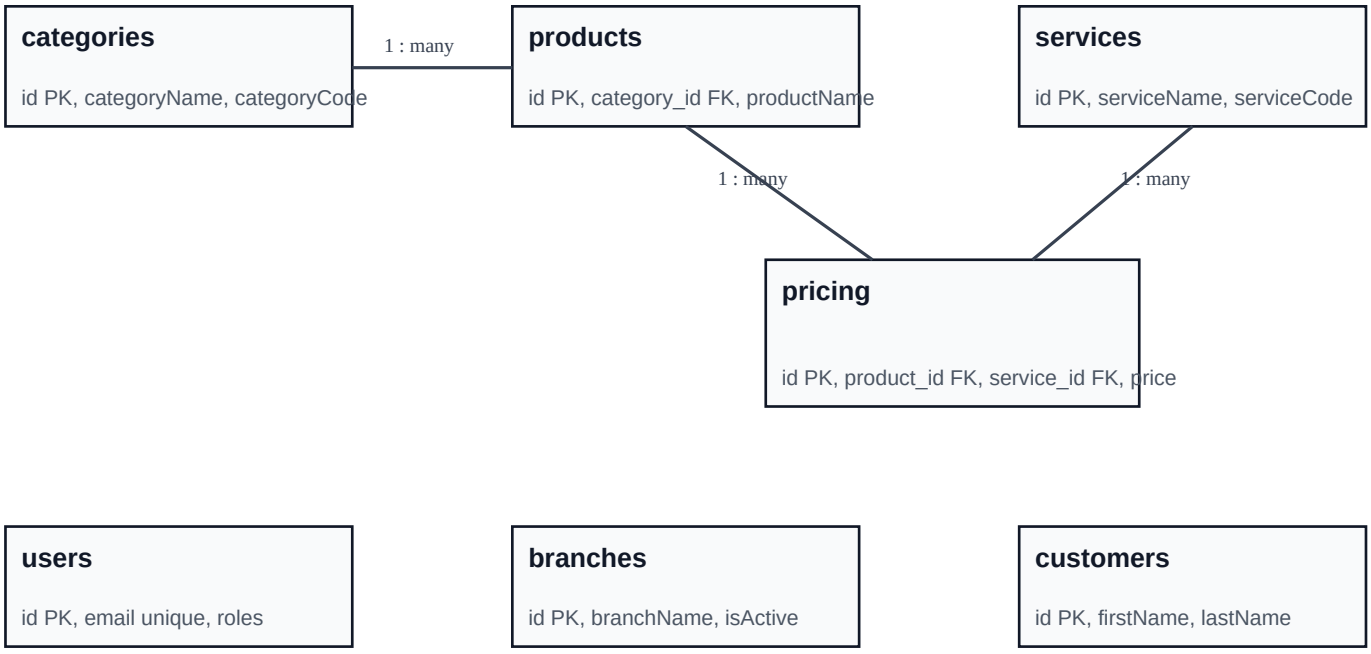
Relationship	Cardinality	Reason
Category -> Product	1 : many	One category contains multiple products
Product -> Pricing	1 : many	A product can have prices for multiple services
Service -> Pricing	1 : many	A service can be applied to multiple products
Product <-> Service	many : many (via Pricing)	Resolved by pricing junction table with unique pair

3. Entity-by-Entity Specification

Table	Key Fields	Notes
users	id, userName, email (unique), password, roles, isActive	Application authentication and authorization
branches	id, branchName, branchAddress, branchPhone, isActive	Operational branch master
customers	id, firstName, lastName, customerPhone, customerEmail (unique), isActive	Customer master
categories	id, categoryName (unique), categoryCode (unique), isActive	Top-level grouping (e.g., regular, hotel, dry wash)
products	id, categoryId (FK), productName, productCode (unique), isActive	Rate-card items such as SHIRT, SAREE, BLANKET
services	id, serviceName (unique), serviceCode (unique), isActive	Service types: WASHING, IRONING, DRY_WASH
pricing	id, productId (FK), serviceId (FK), price, currency, isActive	Active price + amount; unique(productId, serviceId)

4. ER Diagram

Normalized pricing architecture with operational support tables.



Product <-> Service is many-to-many via pricing (unique(product_id, service_id)).

users, branches, and customers are operational master tables for the application.

5. Design Notes & Best Practices

• Keep pricing immutable by versioning (effective_from/effective_to) if you expect frequent price revisions.

• Use pricing.isActive for soft-disable while retaining history.

• Seed services from your sheet as fixed codes: WASHING, IRONING, DRY_WASH.

• Create indexes on pricing(product_id), pricing(service_id), products(category_id).

• For analytics dashboards, build read views joining categories -> products -> pricing -> services.

Source reference: DF SERVICE Google Sheet